details of the error. As discussed previously, you cannot rely on users giving you this information in a timely, accurate manner—if at all. The default logging providers won't give you immediate notification that an error has occurred (unless you spend your day watching a console window on a server), so you need another provider that does. In the next section, you will learn the steps required to create your own logging provider.

12.3.7 Custom email logger

In this section, you will implement a simple logging provider that writes to email. The primary purpose of this exercise is to gain an understanding of the moving parts involved in implementing your own logging provider. For more robust solutions, you should consider implementing a reputable third-party logging solution. Most of them are capable of supporting multiple targets, including email. We will look at implementing one such solution in the next section.

The component responsible for generating the log output implements the `ILogger` interface. The contract defined by the interface requires three methods to be implemented:

- `BeginScope`—This method enables the grouping of a set of logging operations within a scope and sharing of data between the operations. Your logger won't support scopes, but we will explore this feature in more detail later.
- `Log`—This method takes a number of parameters and is responsible for outputting the log.
- `IsEnabled`—This method takes a `LogLevel` as a parameter and determines whether the logger is enabled for the specified `LogLevel`.